

Doc. No.: WG21/P0943R0

Date: 2018-02-11

Reply-to: Hans-J. Boehm

Email: hboehm@google.com

Authors: Hans-J. Boehm

Audience: SG1 (quick check?), LEWG (namespace issues)

P0943R0: Support C atomics in C++

As we previously suggested in [P0063R0](#), it would be nice to have a real interoperability story for C atomics and C++ atomics. In the interest of time, and since it appeared to be less essential and more controversial than the rest of that proposal, this suggestion was not pursued further in later revisions of that paper. Nonetheless, it remains a gaping hole in the interoperability story between C and C++. The solution proposed here is based on one that has been used by Android for several years.

The Problem

It is desirable to make C language header files directly usable by C++ code. It is increasingly common for such header files to include declarations that rely on atomics.

For example, a header may wish to declare a function that provides saturating ("clamped") addition on atomic integers. This can easily be implemented in the common subset of C and C++, except that we have no convenient and portable way to include the definition of names such as `atomic_int`, that are defined in `<atomic>` in C++ and in `<stdatomic.h>` in C.

The best we can currently do in portable code is to use an explicit preprocessor test followed by conditional inclusion of either `<atomic>` or `<stdatomic.h>`. In the former case, we then have to add using declarations to inject the required types and functions into the global namespace. Thus we end up with something like

```
#ifdef __cplusplus
    #include <atomic>
    using std::atomic_int;
    using std::memory_order;
    using std::memory_order_acquire;
    ...
#else /* not __cplusplus */
    #include <stdatomic.h>
#endif /* __cplusplus */
...
int saturated_fetch_add(atomic_int *loc, int value, memory_order order);
```

This approach relies on the currently implicit assumption or wish that the representation of C and C++ atomics are compatible.

Although certainly possible, this is very clumsy compared to the level of interoperability we normally provide; for other language features we commonly provide a C header file that can be included from C++, and provides the necessary declarations. Here we propose to do the same for atomics.

The story here is somewhat confused by the fact that most of the other `.h` compatibility headers currently only appear in the deprecated features section under D.5 C standard library headers [depr.c.headers]. A recent paper, [P0619](#) proposes to undeprecate them. We strongly agree with this paper that the current deprecation of these headers does not reflect reality. Many of them are widely used and heavily relied upon. This proposal assumes that at least some of those headers will be preserved.

History

C and C++ atomics were originally designed together. The original design ensured that the non-generic pieces of the C++ atomics library were usable as a C library. This interoperability story became less clear as a result of several later decisions:

- WG14 adopted, in addition to the non-generic parts of the C++ library, a more generic facility, relying on the `_Atomic` type qualifier, and the `_Atomic(T)` type specifier. This happened late enough in the C++11 standardization cycle that there wasn't much of an opportunity for C++ adjustments.
- C++ has never defined what it means to include `stdatomic.h` in a C++ program.

Shortly before finalizing C++11, we briefly discussed accommodating `_Atomic` roughly as described here.

Proposal

We propose to add a header `stdatomic.h` to the C++ standard. This would mirror the identically named C header, and provide comparable functionality. However, instead of defining it in terms of the C header, we propose that it have the following effects:

1. Include the `<atomic>` header.
2. Define a macro `_Atomic(T)` as `std::atomic<T>`.
3. Promote all `atomic_...` and `memory_order...` identifiers introduced by the `<atomic>` header into the global name space.

Although this provides functionality very similar to the C header, it is unlikely that it will be exactly the same. As usual, it is the responsibility of the author of a C and C++ shared header to ensure that the code is correct in both languages.

This proposal makes no effort to support the C `_Atomic` type qualifier, as opposed to the type *specifier* (which is spelled with parentheses). This is intentional; accommodating the qualifier would be a major language change, particularly since it is the only type qualifier that can affect alignment.

We believe this is the minimum required to conveniently use atomics in shared headers.

Implementation compatibility

This proposal reflects current practice in Android since the Android Lollipop release in 2014. (See <https://android.googlesource.com/platform/bionic/+lollipop-release/libc/include/stdatomic.h>. That uses a single include file shared between C and C++, which may not be the optimal implementation strategy.)

As far as we can tell, it should be easy to support in other implementations. We do rely on the fact that atomics implementations for C and C++ use compatible representation, ordering, and locking conventions. Preliminary experiments with gcc-4.9 suggest that C and C++ atomics generate identical code on x86-64, and code that only gratuitously differs in alignment constraints on ARMv7. In both cases, simple code with mixed atomic operations on objects of different sizes ran correctly.

We were not yet able to reproduce this experiment with other compilers, in part because support for C atomics on older, default installed, compiler versions is often still lacking. We hope to test with newer compilers soon.

There is no reason to implement C and C++ atomics differently. Implementing ordering constraints in incompatible ways is already greatly undesirable, in that mixed language programs with `memory_order` annotations would no longer be sequentially consistent. Using incompatible locking schemes for large atomics would generally double the amount of space required for the lock table, and complicate ABI conventions, with no benefit.

Thus we expect that in most cases, the implementation effort here consists of adding a simple header file. Implementations that also support C atomics should already satisfy the remaining constraints. And those constraints are vacuous for implementations that don't.

Should there be a `<cstdatomic>`?

We do not believe there is a strong need to introduce a `<cstdatomic>` that only introduces names into the `std::` namespace. The justification for `<stdatomic.h>` is entirely to support headers shared between C and C++, not to import a C facility into C++. A shared header will not be able to take advantage of names introduced into the `std::` namespace.

There is an argument that `<cstdatomic>` should be provided anyway for uniformity. It's not clear that this outweighs the cost of introducing a header that we expect to get essentially no use.

Wording

Placement and details of the wording are heavily dependent on the outcome of the P0619 discussion. We expect something along the following lines:

The header `<stdatomic.h>` behaves as if it consisted of:

```
#include <atomic>  
#define Atomic(T) std::atomic<T>  
using std::memory_order;  
using std::atomic X; // Repeated for all X  
using std::memory_order Y; // Repeated for all Y
```

where the last two using directives are repeated for every identifier with names starting with `atomic` or `memory_order` introduced by Clause 32 [atomics].

[Note: Implementations should ensure that C and C++ representations of atomic objects are compatible, so that the same object can be safely accessed as both an `Atomic(T)` from C code and `atomic<T>` from C++ code. The representations should be the same, and the mechanisms used to ensure atomicity and memory ordering should be compatible. --end note]

Note that the above is essentially a verbatim reflection of the part of the Android `<stdatomic.h>` header that is used when compiling C++.